ACTIVIDAD PRACTICA

Diseño arquitectural del sistema con IA

Propuesta arquitectónica - EduTech Academy

Documento de arquitectura elaborado a partir del análisis de negocio de la Unidad 2. La propuesta mantiene el alcance de EduTech Academy y organiza dominio, aplicación, persistencia, transacciones, CQRS, revisión SOLID y decisiones arquitectónicas.

Empresa asignada	**EduTech Academy**
Integrantes	Carlos Hugo García Gómez – 2226690 Alan Bertín Castillo Martínez – 2226653 Saúl Alejandro Hernández Garza – 2226664
Fecha de entrega	14/10/2026

Propósito de la actividad

El objetivo consiste en organizar las reglas, invariantes y responsabilidades operativas de EduTech Academy antes de su codificación completa. La propuesta demuestra la transición desde los productos de la Unidad 2 hacia una arquitectura que separa con claridad el dominio, la aplicación y la infraestructura.

Continuidad con la Unidad 2

Se mantiene la misma empresa y el alcance aprobado. Los productos de la Unidad 2 se utilizan como evidencia para justificar cada decisión de la arquitectura, evitando incorporar reglas que no hayan sido identificadas o marcadas como pendientes de validación.

Insumo de la Unidad 2	Como se utilizara en la Unidad 3
BRD	Delimitar alcance, actores, necesidades, restricciones, supuestos y criterios de exito.
Historias de usuario y Gherkin	Identificar casos de uso, condiciones de negocio y comportamientos esperados.
BPMN	Reconocer la secuencia, decisiones, participantes, excepciones y puntos de coordinacion.
Glosario y modelo de dominio	Mantener un lenguaje ubicuo y localizar conceptos con identidad, valor o ciclo de vida.
Matriz de trazabilidad	Relacionar cada decision arquitectural con una necesidad, historia, regla o escenario.
Repositorio de GitHub	Conservar versiones, diagramas, documentos, prompts y codigo generado o revisado.

Etapa 1 Preparar el caso

Se seleccionan cinco casos de uso vinculados con el proceso central de EduTech Academy. El conjunto incluye consultas, operaciones que modifican estado y escenarios que pueden ser rechazados por reglas de cupo, autorización o certificación.

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Consulta del catálogo de cursos	Estudiante	US-01	El sistema muestra los cursos disponibles con información consistente de modalidad, calendario y estado.
CU-02	Inscripción individual o empresarial	Estudiante / Cliente empresarial	US-02, US-03	Se genera la inscripción, se ocupa un lugar del grupo y se crea el pago pendiente; si no hay cupo, se rechaza.
CU-03	Gestión de curso y control de cupo	Administración	US-04, US-17	El curso controla mínimo 5 y máximo 25 inscritos, cierra inscripción y permite sustituir instructor conservando el historial.
CU-04	Registro académico y modificación de calificación	Instructor / Control Escolar	US-07, US-08, US-09	Se registran asistencias y calificaciones; cualquier corrección autorizada conserva motivo y trazabilidad.
CU-05	Validación y emisión de certificado	Control Escolar	US-10, US-11	El estudiante se marca Apto solo si cumple calificación, asistencia, curso concluido y pago regularizado; después se emite el certificado.

Etapa 2 Elegir el patrón de lógica de negocio

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Baja. Las reglas terminan repetidas en procedimientos largos.	Media. Funciona bien con datos tabulares, pero distribuye reglas entre tablas.	Alta y encapsulada. Las reglas viven en Curso, Inscripción, RegistroAcademico, Certificado y sus objetos de valor.	BRD/US: certificación con cuatro condiciones, cupo 5–25, estados, pagos, cambios de grupo y excepciones.
Cambios esperados	Alta fricción: un cambio obliga a modificar varios scripts.	Moderada: los cambios son sencillos si dependen de una sola tabla.	Baja fricción: cada política se cambia en el objeto del dominio correspondiente.	Reglas aún por definir: asistencia mínima, plazo de pago, costo de reinscripción y límite de cambios de grupo.
Interacción entre conceptos	Débil: datos y comportamiento quedan separados.	Basada en tablas, adecuada para consultas pero limitada para ciclos de vida.	Rica y colaborativa: Curso, Grupo, Inscripción, Pago, RegistroAcademico y Certificado colaboran sin perder sus responsabilidades.	Mapa de dominio y lenguaje ubicuo de EduTech Academy.
Estados e invariantes	Las transiciones válidas deben repetirse en cada script.	Puede validar columnas, pero no protege bien reglas cruzadas.	Cada aggregate protege sus invariantes y rechaza transiciones inválidas.	Estados de curso, inscripción, pago, certificación y solicitud de cambio de grupo.
Criterio adicional: trazabilidad y auditoría	La auditoría y las excepciones se dispersan en condicionales.	La trazabilidad queda ligada a tablas o triggers.	Domain Events y Bitácora permiten trazabilidad y coordinación desacoplada.	RNF-03 y eventos como CalificacionModificada, CursoCancelado y CertificadoEmitido.

Justificación de la decisión

Justificación arquitectónica: elección de Domain Model para EduTech Academy

La elección de Domain Model para EduTech Academy permite organizar las reglas de inscripción, cursos, pagos, registro académico, certificación, cambios de grupo y cancelaciones alrededor de objetos del negocio. Este enfoque reduce la repetición de reglas, protege los estados válidos y facilita modificar políticas que todavía no han sido definidas por la organización.

1. Por qué se descartan los patrones alternativos

- Por qué se descarta Transaction Script: aunque es sencillo para operaciones pequeñas, en EduTech provocaría que reglas como cupo, certificación, pagos y cambios de estado se repitieran en distintos procedimientos. Cada nueva excepción aumentaría el número de condicionales y el riesgo de inconsistencias.
- Por qué se descarta Table Module: organiza bien operaciones tabulares y reportes, pero las reglas de EduTech atraviesan Curso, Inscripción, Pago, Registro Académico y Certificado. La lógica quedaría repartida entre módulos ligados a tablas y sería más difícil proteger invariantes y ciclos de vida.

2. Problemas concretos que resuelve Domain Model

La elección de Domain Model responde a los problemas identificados en EduTech Academy: validación manual de certificados, información de asistencia y calificaciones separada, dificultad para rastrear cambios de grupo y necesidad de mantener reglas consistentes.

- Protección de reglas y estados: cada aggregate protege sus propias invariantes. Curso y Grupo controlan el cupo; Inscripción y Pago evolucionan de forma coordinada; RegistroAcademico concentra asistencia y calificaciones; Certificado valida los requisitos de certificación.

- Manejo de reglas de negocio y excepciones: cancelación de cursos, cambio de grupo, reprobación, reinscripción y sustitución de instructor se modelan como comportamiento del dominio y Domain Events, evitando dispersar decisiones en la interfaz o en scripts independientes.
- Control de autorizaciones y trazabilidad: operaciones como modificar una calificación o autorizar un cambio de grupo deben conservar quién realizó y autorizó el cambio. La Bitácora registra los cambios relevantes de forma inmutable.

3. Costos introducidos por la decisión

- Curva de aprendizaje y complejidad de diseño: Domain Model exige definir aggregates, entidades, value objects, estados y eventos antes de desarrollar. El costo inicial es mayor que en patrones procedurales.
- Mayor trabajo inicial de implementación: la arquitectura necesita Service Layer y Repositories para coordinar casos de uso y persistencia. Se acepta este costo porque permite centralizar reglas y adaptar con menor impacto las políticas que EduTech aún debe confirmar.

Etapa 3 Construir el mapa de dominio

El mapa de dominio traduce el lenguaje y las reglas de la Unidad 2 a elementos con responsabilidades explícitas. Se distinguen entidades, Value Objects, Aggregates y Domain Events, manteniendo el lenguaje ubicuo de EduTech Academy.

Elemento	Pregunta que debe responder el equipo	Respuesta del equipo
Lenguaje ubicuo	Terminos usados por usuarios, analistas y desarrolladores con el mismo significado.	Estudiante, Curso, Grupo, Cupo, Inscripción, Pago pendiente, Registro académico, Evaluación, Calificación, Asistencia, Apto para certificación, Certificado, Cambio de grupo, Cancelación de curso, Reembolso, Reinscripción y Bitácora.
Entidades	Conceptos con identidad y ciclo de vida que deben distinguirse aun cuando cambien sus datos.	Curso; Grupo; Evaluacion; Inscripcion; Pago; RegistroAcademico; Asistencia; Calificacion; Certificado; SolicitudCambioGrupo; CancelacionCurso; CasoResolucion; Estudiante; ClienteEmpresarial; Instructor; RegistroBitacora.
Value Objects	Conceptos definidos por sus valores y reglas, sin identidad propia relevante.	Dinero, Modalidad, CalendarioCurso, PeriodoInscripcion, Cupo, TipoEvaluacion, Nota, PorcentajeAsistencia, MotivoCorreccion, Rol y los estados de curso, inscripción, pago, certificación, solicitud y caso de resolución.
Aggregates	Grupos que requieren un limite de consistencia y una raiz para controlar cambios.	Curso; Inscripcion; RegistroAcademico; Certificado; SolicitudCambioGrupo; CancelacionCurso; Estudiante; ClienteEmpresarial; Instructor; Bitacora. Cada raíz protege sus invariantes y referencia otros aggregates por ID.
Domain Services	Comportamientos del dominio que no pertenecen naturalmente a una sola entidad.	La arquitectura prioriza comportamiento dentro de los aggregates. Las operaciones que coordinan varios aggregates se ubican en la Service Layer y las reglas pendientes se mantienen como parámetros configurables.
Domain Events	Hechos del negocio que ya ocurrieron y pueden interesar a otros componentes.	InscripcionRealizada, CupoMaximoAlcanzado, PagoRegistrado, PlazoDePagoVencido, AsistenciaRegistrada, CalificacionModificada, EstudianteMarcadoApto, CertificadoEmitido, CambioGrupoAutorizado, CursoCancelado, ReembolsoSolicitado, EstudianteReinscritoEnOtroCurso, InstructorSustituido y ReinscripcionRealizada.

Diagrama del mapa de dominio

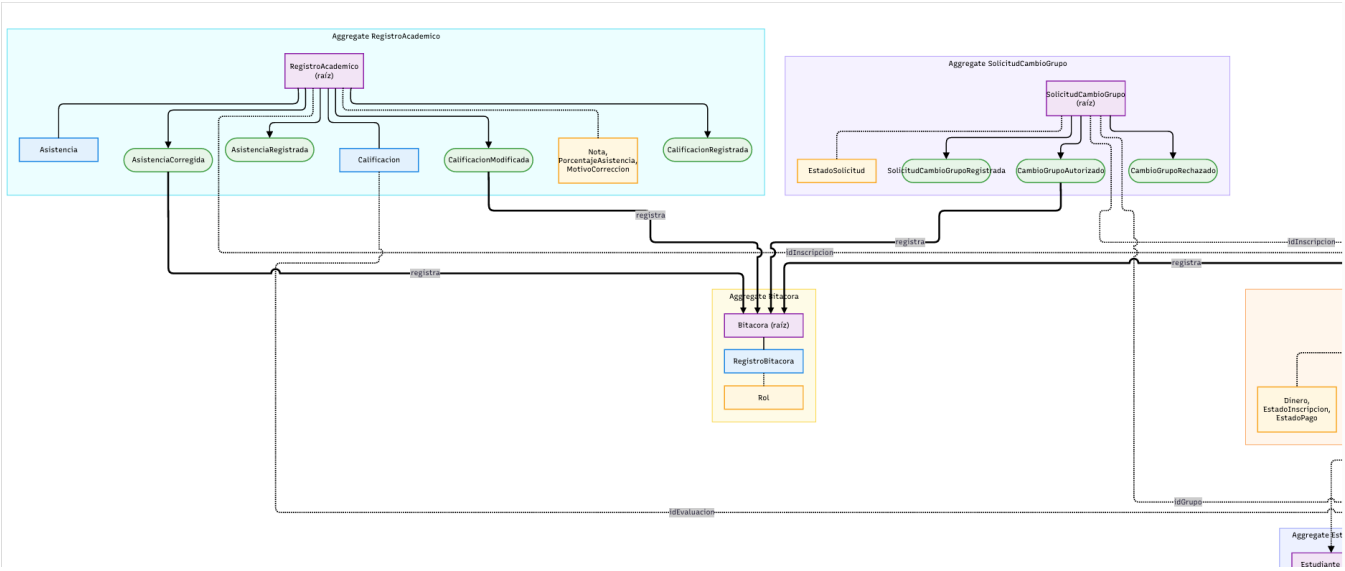


Diagrama de dominio de EduTech Academy - vista 1 de 4.

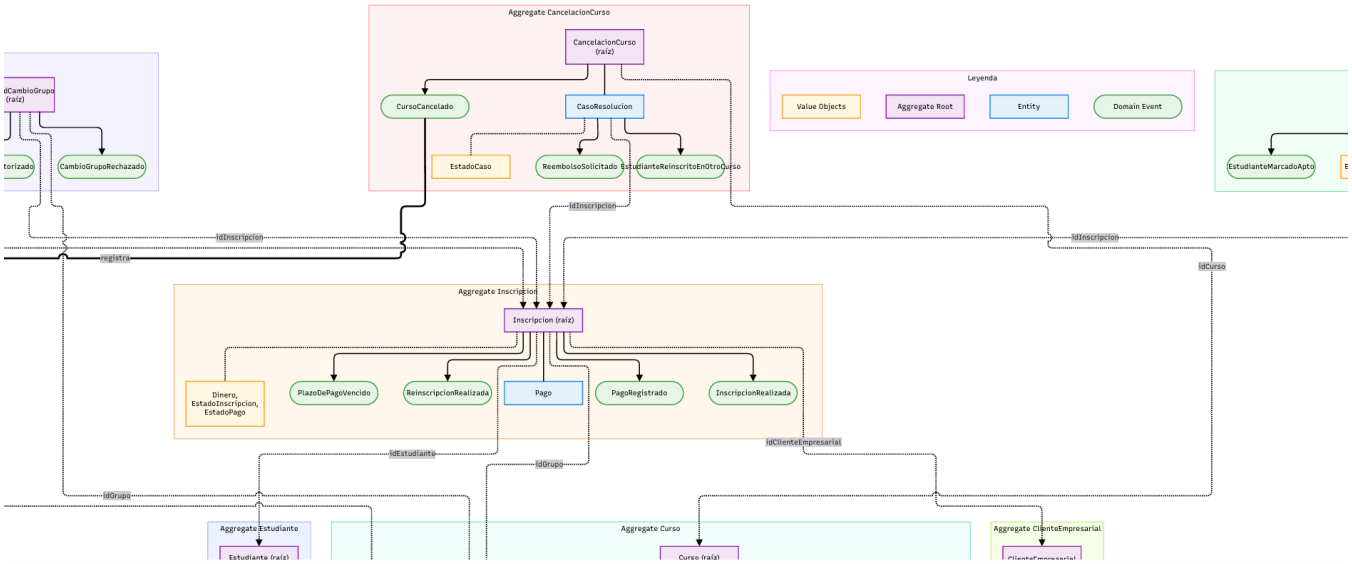


Diagrama de dominio de EduTech Academy - vista 2 de 4.

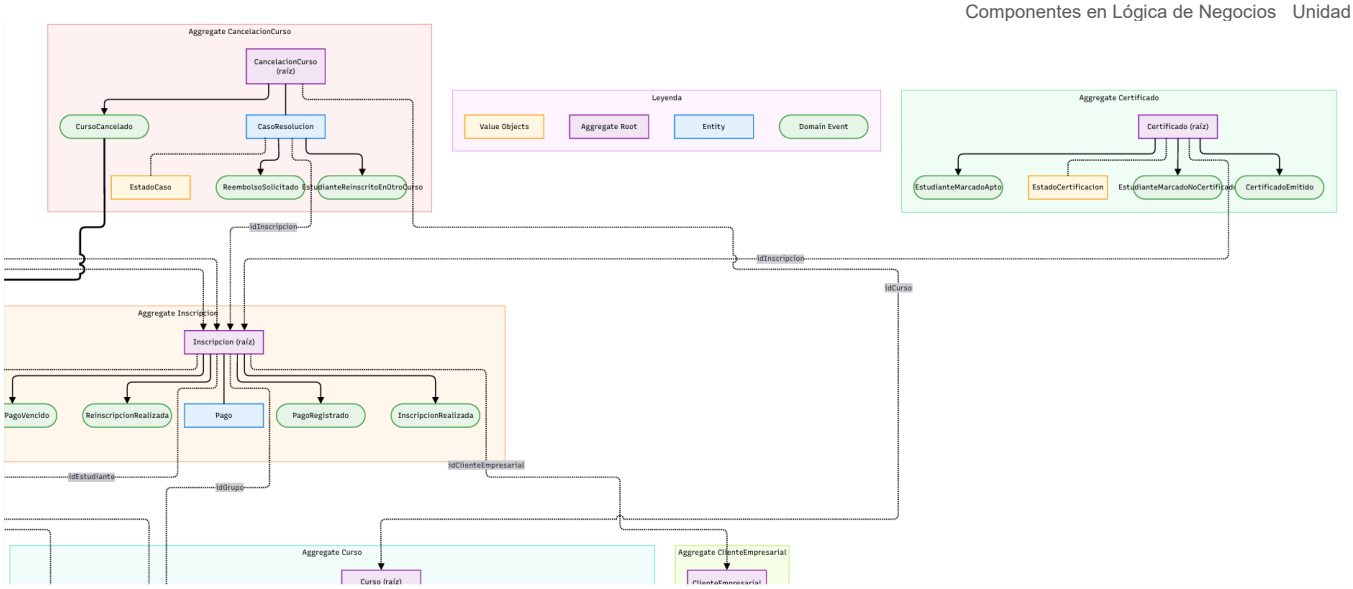


Diagrama de dominio de EduTech Academy - vista 3 de 4.

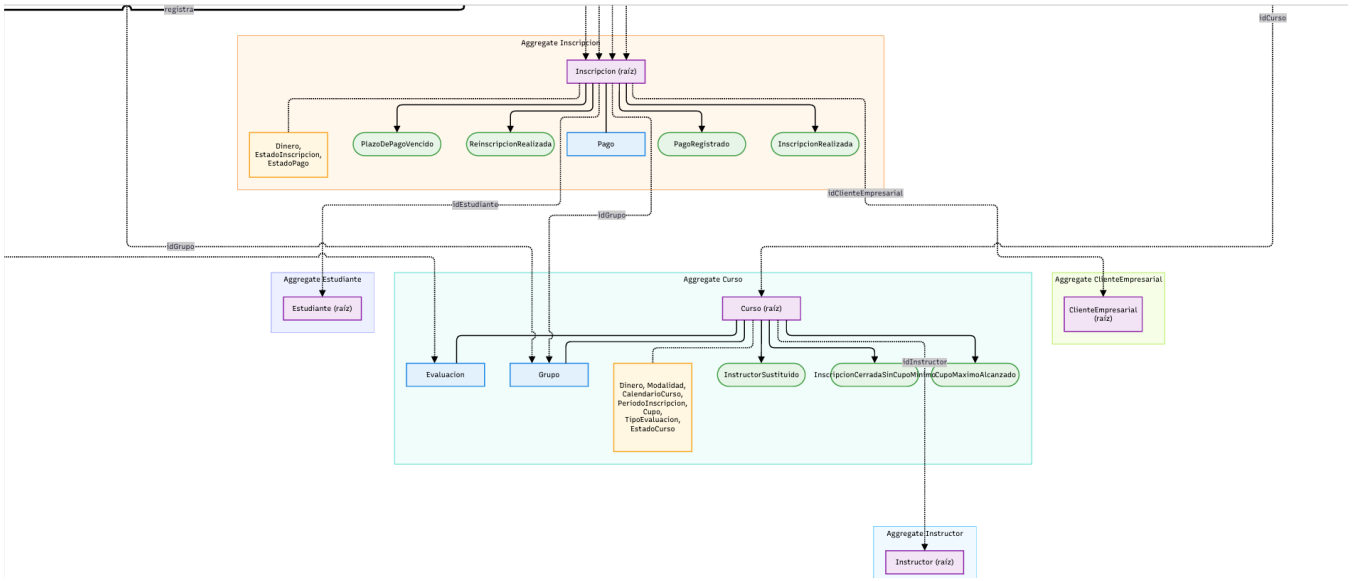


Diagrama de dominio de EduTech Academy - vista 4 de 4.

Glosario actualizado

Término	Definición en el lenguaje ubicuo del equipo	Estado
Aggregate / Aggregate Root	Grupo de objetos que se trata como una unidad de consistencia. Toda modificación interna pasa por su raíz.	Nuevo
Value Object	Concepto definido por sus valores y sin identidad propia; por ejemplo Cupo, Dinero o Nota.	Nuevo
Domain Event	Hecho del negocio que ya ocurrió y que puede provocar otras acciones, por ejemplo PagoRegistrado o CursoCancelado.	Nuevo
Application Service	Orquestador de un caso de uso. Obtiene aggregates, invoca el dominio y coordina persistencia sin contener reglas propias.	Nuevo
Repository	Abstracción para recuperar y guardar aggregate roots sin acoplar el dominio a SQL o a tablas.	Nuevo
Unit of Work	Agrupa los cambios de un caso de uso en una transacción atómica: se guardan todos o ninguno.	Nuevo
Invariante	Regla que un aggregate debe proteger siempre; por ejemplo, un Grupo no puede superar el cupo máximo.	Nuevo
Command / Query	Command modifica estado; Query consulta sin modificarlo. Se separan cuando aporta claridad al caso de uso.	Nuevo
Curso	Servicio educativo con instructor, grupos, evaluaciones, calendario, modalidad, precio y estado.	Corregida
Grupo	Sección de un curso con identidad y cupo propio.	Nuevo
Inscripción	Relación entre estudiante, curso y grupo; mantiene su estado y contiene el pago.	Corregida
Registro académico	Aggregate que reúne asistencia y calificaciones de una inscripción.	Nuevo
Apto para certificación	Estado del estudiante que cumple los criterios de calificación, asistencia, curso concluido y pago regularizado.	Corregida
Cancelación de curso	Decisión que genera un caso de resolución para cada persona inscrita.	Nuevo
Caso de resolución	Seguimiento individual de reembolso o reinscripción cuando un curso es cancelado.	Nuevo
Bitácora	Historial inmutable de cambios relevantes: quién, qué, cuándo, valor anterior, valor nuevo y quién autorizó.	Corregida

Fichas técnicas de los Aggregates

Aggregate (raíz)	Responsabilidades	Invariantes que protege	Eventos posibles
Curso (raíz) Contiene: Grupo, Evaluacion	Publicar el curso, cerrar la inscripción, verificar cupo, asignar/sustituir instructor y cancelar.	El cupo (mínimo 5, máximo 25) no se rompe; el curso tiene un estado válido (Disponible, Sin cupo disponible, En curso, Cancelado).	CupoMaximoAlcanzado, InscripcionCerradaSinCupoMinimo, InstructorSustituido
Inscripcion (raíz) Contiene: Pago	Registrar pago, marcar riesgo de pérdida de lugar, cambiar de grupo y reinscribir.	La inscripción y su pago cambian de estado juntos (Inscrito, Pago pendiente, Pagado).	InscripcionRealizada, PagoRegistrado, PlazoDePagoVencido, ReinscripcionRealizada
RegistroAcademico (raíz) Contiene: Asistencia, Calificacion	Registrar asistencia y calificaciones, calcular el porcentaje de asistencia y evaluar si se cumplen los requisitos académicos.	Asistencias y calificaciones ligadas al estudiante y curso correcto y evaluables juntas; una calificación solo se modifica con autorización (US-09) y una asistencia cerrada solo se corrige con motivo (US-07).	AsistenciaRegistrada, AsistenciaCorregida, CalificacionRegistrada, CalificacionModificada
Certificado (raíz)	Validar los requisitos de certificación y emitir el certificado.	El certificado solo se emite si se cumplen todos los requisitos (calificación mínima 70, asistencia mínima, curso concluido y pago regularizado).	EstudianteMarcadoApto, EstudianteMarcadoNoCertificado, CertificadoEmitido
SolicitudCambioGrupo (raíz)	Registrar, autorizar o rechazar el cambio de grupo de un estudiante.	El cambio sigue su flujo: Pendiente de autorización, Autorizada o Rechazada.	SolicitudCambioGrupoRegistrada, CambioGrupoAutorizado, CambioGrupoRechazado
CancelacionCurso (raíz) Contiene: CasoResolucion	Registrar la cancelación (motivo, fecha, quién autorizó) y generar los casos de resolución.	Cada inscrito de un curso cancelado tiene un caso de resolución (EX-01) con reembolso o reinscripción.	CursoCancelado, ReembolsoSolicitado, EstudianteReinscritoEnOtroCurso
Estudiante (raíz)	Mantener la identidad de la persona que se inscribe, paga, asiste y es evaluada.	Se identifica por su matrícula o ID, no por sus datos.	Sin eventos propios.
ClienteEmpresarial (raíz)	Mantener la identidad de la empresa que inscribe a varios empleados.	Se identifica por su ID, no por sus datos.	Sin eventos propios.
Instructor (raíz)	Mantener la identidad de quien imparte el curso; puede ser sustituido a mitad de curso.	Se identifica por su ID, no por sus datos.	Sin eventos propios (InstructorSustituido lo lanza Curso).
Bitacora (raíz) Contiene: RegistroBitacora	Registrar quién, qué, cuándo, valor anterior, valor nuevo y quién autorizó cada cambio importante.	Todo cambio importante queda registrado y no se modifica después (RNF-03).	Sin eventos propios: registra los eventos de los demás aggregates.

Decisiones dudosas o descartadas

Pregunta	Duda	Decisión
¿Calificación es Entidad o Value Object?	La nota es un valor, pero la calificación registrada puede corregirse y requiere historial.	Se modela Calificacion como entidad dentro de RegistroAcademico y Nota como Value Object.
¿Grupo debe ser Aggregate Root?	Tiene identidad y cupo propio, pero no tiene sentido fuera de su Curso.	Se mantiene como entidad dentro del aggregate Curso; las operaciones pasan por Curso.
¿Cupo es entidad o Value Object?	Tiene mínimo, máximo y lugares ocupados, pero no tiene identidad propia.	Se modela como Value Object dentro de Grupo: solo importa por sus valores (mínimo 5, máximo 25 y ocupados).
¿Pago debe ser un aggregate independiente?	Tiene identidad y estado, pero su ciclo de vida está directamente ligado a la Inscripción.	Se mantiene como entidad dentro del aggregate Inscripcion.
¿Bitácora debe pertenecer a cada aggregate?	La usan múltiples procesos y sus registros deben ser inmutables.	Se mantiene como aggregate independiente Bitacora con RegistroBitacora.
¿Las autorizaciones pertenecen a aplicación o dominio?	Hay permisos de acceso y también reglas de negocio que requieren autorización.	La Service Layer verifica quién invoca; el dominio protege la invariante y rechaza operaciones sin una autorización válida.

Etapa 4 Diseñar la Service Layer

La Service Layer organiza la coordinación de los casos de uso. Cada Application Service recibe la solicitud, recupera los aggregates necesarios mediante Repository, invoca el comportamiento del dominio, coordina la persistencia y devuelve el resultado, sin absorber las reglas que pertenecen al Domain Model.

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
CatalogoService	consultarCatalogo	Filtros opcionales de curso/modalidad	CursoRepository	Catálogo de cursos	Solo coordina la lectura; las reglas de disponibilidad permanecen en Curso.
InscripcionService	registrarInscripcion	estudianteId, cursoId, grupoId	EstudianteRepository, CursoRepository, InscripcionRepository, UnitOfWork	Inscripción creada	Curso/Grupo valida periodo y cupo; Inscripción genera Pago pendiente.
CursoService	cerrarInscripcion	cursoId	CursoRepository, aggregate Curso, UnitOfWork, EventBus	Curso actualizado	Curso verifica cupo mínimo/máximo y emite el evento correspondiente.
AcademicoService	registrarCalificacion	inscripcionId, evaluacionId, nota	RegistroAcademicoRepository, aggregate RegistroAcademico, UnitOfWork	Registro académico actualizado	Nota valida rango y RegistroAcademico protege la relación con estudiante/curso.
CertificacionService	validarRequisitos	inscripcionId	AccesoService, RegistroAcademicoRepository, InscripcionRepository, CursoRepository, CertificadoRepository, UnitOfWork, Bitacora	Apto para certificación o No certificado con motivo	Certificado valida calificación, asistencia, curso concluido y pago regularizado.
CertificacionService	emitirCertificado	inscripcionId	AccesoService, CertificadoRepository, UnitOfWork, Bitacora	Certificado emitido o rechazo	Certificado solo se emite si el estudiante está Apto para certificación.
CancelacionService	cancelarCurso	cursoId, motivo, autorizadoPor	AccesoService, CursoRepository, CancelacionRepository, InscripcionRepository, UnitOfWork, EventBus, Bitacora	Cancelación y casos de resolución	El dominio cancela el curso y crea un CasoResolucion por cada inscripción afectada.
CancelacionService	resolverCasoCancelacion	inscripcionId, opción elegida (reembolso o reinscripción), cursoId alterno	CursoRepository, CancelacionRepository, InscripcionRepository, UnitOfWork, EventBus, Bitacora	Reembolso solicitado o reinscripción completada	CasoResolucion cambia de estado; Grupo valida el cupo del curso alterno; el pago se traslada a la nueva Inscripción.

Revisión del Service Layer

- ¿El nombre del método expresa una acción del negocio?

Respuesta del equipo: Sí. Los métodos usan verbos del lenguaje de EduTech: consultarCatalogo, registrarInscripcion, cerrarInscripcion, registrarCalificacion, validarRequisitos, emitirCertificado y cancelarCurso.

- ¿El método coordina o contiene decisiones que deberían estar en el dominio?

Respuesta del equipo: Los Application Services coordinan el flujo, obtienen aggregates mediante repositorios y persisten resultados. Las decisiones sobre cupo, estados, certificación y correcciones permanecen dentro del Domain Model.

- ¿El servicio depende de interfaces o de detalles concretos de base de datos?

Respuesta del equipo: Depende de abstracciones como CursoRepository, InscripcionRepository, RegistroAcademicoRepository y UnitOfWork; no depende directamente de SQL ni de tablas.

- ¿La operación tiene un inicio y un final transaccional comprensible?

Respuesta del equipo: Sí. Cada caso de uso agrupa sus cambios mediante Unit of Work, de manera que la operación se confirma completa o se revierte ante un fallo.

- ¿Los errores del dominio pueden explicarse con términos del negocio?

Respuesta del equipo: Sí. Los rechazos se expresan con el lenguaje ubicuo, por ejemplo: “Grupo sin cupo”, “Pago pendiente”, “Estudiante no apto para certificación” o “Cambio de grupo no autorizado”.

Etapa 5 Separar persistencia y transacciones

Los repositorios se expresan en términos del dominio y abstraen el mecanismo concreto de persistencia. El límite transaccional se define mediante Unit of Work para que los cambios de cada caso de uso se confirmen como una sola unidad o se reviertan ante un error.

Repository	Necesidad del dominio	Metodos propuestos	Objeto que recupera o guarda
CursoRepository	Recuperar y guardar Curso protegiendo grupos, evaluaciones, cupo y estado.	findById(cursoId): Curso save(curso): void	Aggregate Curso con Grupo y Evaluacion.
InscripcionRepository	Gestionar el ciclo de vida de Inscripcion y Pago.	findById(inscripcionId): Inscripcion findByEstudiante(estudianteId): List<Inscripcion> save(inscripcion): void	Aggregate Inscripcion con Pago.
RegistroAcademicoRepository	Recuperar el expediente académico de una inscripción.	findByInscripcion(inscripcionId): RegistroAcademico save(registro): void	Aggregate RegistroAcademico con Asistencia y Calificacion.
CertificadoRepository	Recuperar y guardar el Certificado de una inscripción con su estado de certificación.	findByInscripcion(inscripcionId): Certificado save(certificado): void	Aggregate Certificado.
CancelacionRepository	Guardar la cancelación de un curso con los casos de resolución de cada inscrito.	findByCurso(cursoId): CancelacionCurso save(cancelacion): void	Aggregate CancelacionCurso con CasoResolucion.
EstudianteRepository	Recuperar al estudiante que solicita la inscripción.	findById(estudianteId): Estudiante	Aggregate Estudiante.

Alcance transaccional (Unit of Work)

Dónde inicia la transacción: en el Application Service, al comenzar el caso de uso y después de verificar el permiso del rol y recuperar los aggregates necesarios mediante los repositorios.

Qué cambios incluye: todos los cambios del caso de uso se confirman juntos. Por ejemplo, en cancelarCurso se guardan el Curso cancelado y la CancelacionCurso con un CasoResolucion por inscrito, y en emitirCertificado el Certificado; en ambos casos se incluye el RegistroBitacora correspondiente.

Qué ocurre ante un error: si el dominio rechaza la operación (por ejemplo, “Grupo sin cupo” o “Estudiante no apto para certificación”) o falla la persistencia, la transacción se revierte: se guardan todos los cambios o ninguno.

Etapa 6 Separar Commands y Queries

Se aplica una separación CQRS básica cuando aporta claridad: los Commands cambian el estado del sistema y las Queries recuperan información sin modificarlo. No se plantea una separación física obligatoria de bases de datos.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	RegistrarInscripcionCommand	estudianteId, cursoId, grupoId	RegistrarInscripcionHandler (CursoRepository, InscripcionRepository, UnitOfWork, EventBus)	Inscripción creada y pago pendiente	Modifica el estado, ocupa cupo y emite InscripcionRealizada.
Command	EmitirCertificadoCommand	inscripcionId	EmitirCertificadoHandler (RegistroAcademicoRepository, InscripcionRepository, CertificadoRepository, UnitOfWork)	Certificado emitido o rechazo	Modifica el estado de certificación y exige cumplir todos los criterios.
Query	ConsultarCatalogoQuery	Filtros opcionales	ConsultarCatalogoHandler (CursoRepository)	Lista de cursos (CursoDisponibleDTO)	Solo lectura; no modifica aggregates ni requiere transacción de escritura.
Query	ConsultarPagosPendientesQuery	Filtros de periodo/curso	ConsultarPagosPendientesHandler (InscripcionRepository)	Lista de PagoPendienteDTO	Solo lectura para Administración; no cambia el estado del dominio.

RegistrarInscripcionCommand: modifica el estado del sistema al crear una inscripción, ocupar cupo y generar un pago pendiente. Handler: RegistrarInscripcionHandler. Dependencias: CursoRepository, InscripcionRepository, UnitOfWork, EventBus.

EmitirCertificadoCommand: modifica el estado de certificación al emitir el documento para un estudiante apto. Handler: EmitirCertificadoHandler. Dependencias: RegistroAcademicoRepository, InscripcionRepository, CertificadoRepository, UnitOfWork.

- Seleccionen al menos dos necesidades de información y descríbanlas como Queries.

ConsultarCatalogoQuery: consulta los cursos sin alterar el estado del sistema. Handler: ConsultarCatalogoHandler. Dependencias: CursoRepository.

ConsultarPagosPendientesQuery: consulta inscripciones con pago pendiente. Handler: ConsultarPagosPendientesHandler. Dependencias: InscripcionRepository.

- Definan el Handler de cada operación y sus dependencias.

Cada Command y cada Query tiene su propio Handler con sus dependencias, indicados arriba y en la tabla.

- Diseñen el DTO o View Model de lectura sin reutilizarlo automáticamente como modelo de escritura.

CursoDisponibleDTO: estructura de lectura con curso, modalidad, calendario y disponibilidad, sin exponer el aggregate completo.

PagoPendienteDTO: estructura de lectura con inscripción, estudiante, curso, monto y estado de pago.

- Justifiquen si alguna parte del caso no necesita esta separación.

No es necesario aplicar CQRS de forma estricta a validaciones internas del mismo aggregate ni a operaciones auxiliares simples. La separación se usa donde distingue claramente lectura y escritura sin añadir complejidad innecesaria.

Etapa 7 Revisar la propuesta con SOLID

La revisión SOLID se utiliza para detectar responsabilidades mal ubicadas, acoplamiento innecesario y contratos demasiado amplios. Los hallazgos se documentan con evidencia, impacto y decisión del equipo.

Principio	Evidencia encontrada	Impacto	Decisión confirmada
S	Los Application Services coordinan casos de uso; Curso, Inscripcion, RegistroAcademico y Certificado protegen reglas propias.	Evita servicios monolíticos y aggregates con responsabilidades ajenas.	Se mantiene la separación entre coordinación y lógica de negocio.
O	Value Objects y reglas configurables permiten cambiar asistencia mínima, plazos o políticas sin dispersar if.	Facilita incorporar políticas futuras.	Las reglas variables se encapsulan y parametrizan.
L	Los repositorios se definen como contratos que pueden tener implementación real o en memoria.	Facilita pruebas y cambios de infraestructura.	Se mantienen interfaces para repositorios.
I	CursoRepository, InscripcionRepository y RegistroAcademicoRepository se separan por aggregate.	Reduce acoplamiento innecesario.	Se evitan repositorios genéricos gigantes.
D	Los servicios dependen de Repository y UnitOfWork, no de SQL/ORM.	Protege el núcleo ante cambios tecnológicos.	Infraestructura implementa las abstracciones definidas por la arquitectura.

Etapa 8 Representar y documentar la arquitectura

Diagrama de clases DDD

El diagrama de clases representa los aggregates, entidades, Value Objects y relaciones relevantes de EduTech Academy. Para mantener legibilidad se presenta en cuatro vistas.

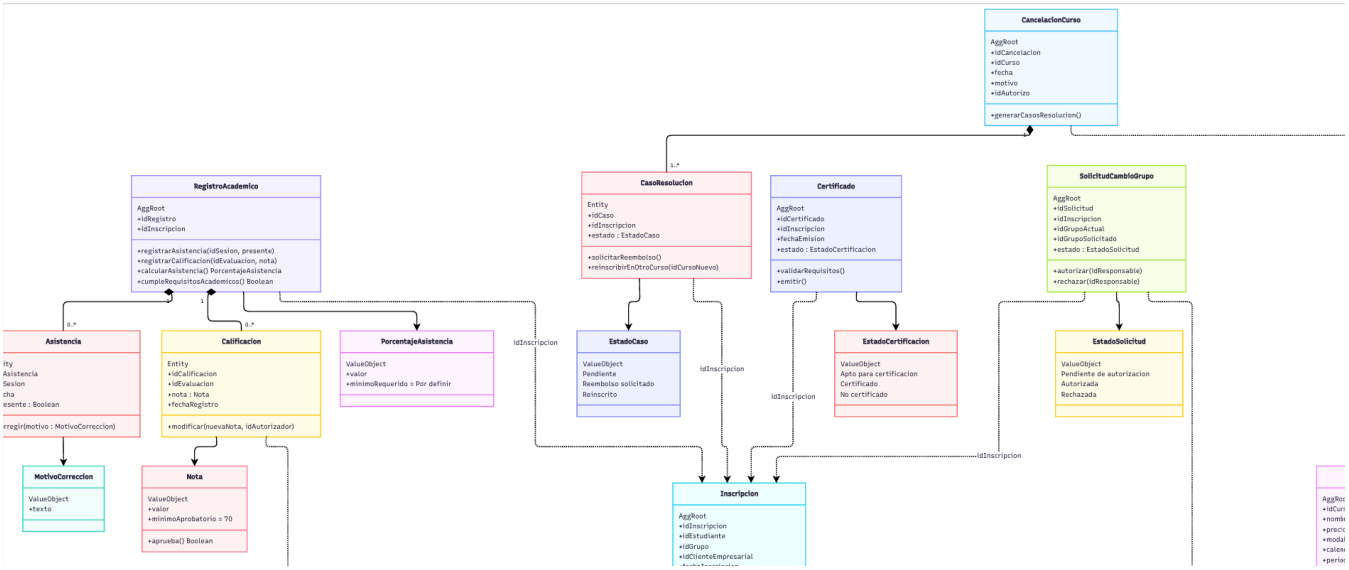


Diagrama de clases DDD de EduTech Academy - vista 1 de 4.

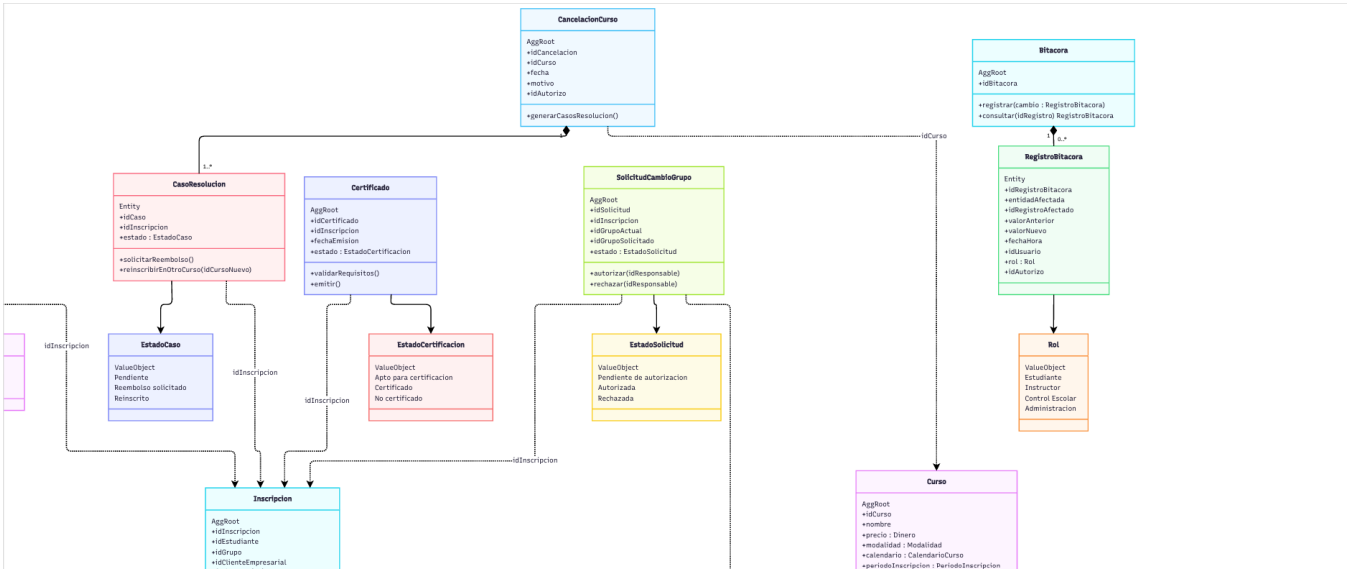


Diagrama de clases DDD de EduTech Academy - vista 2 de 4.

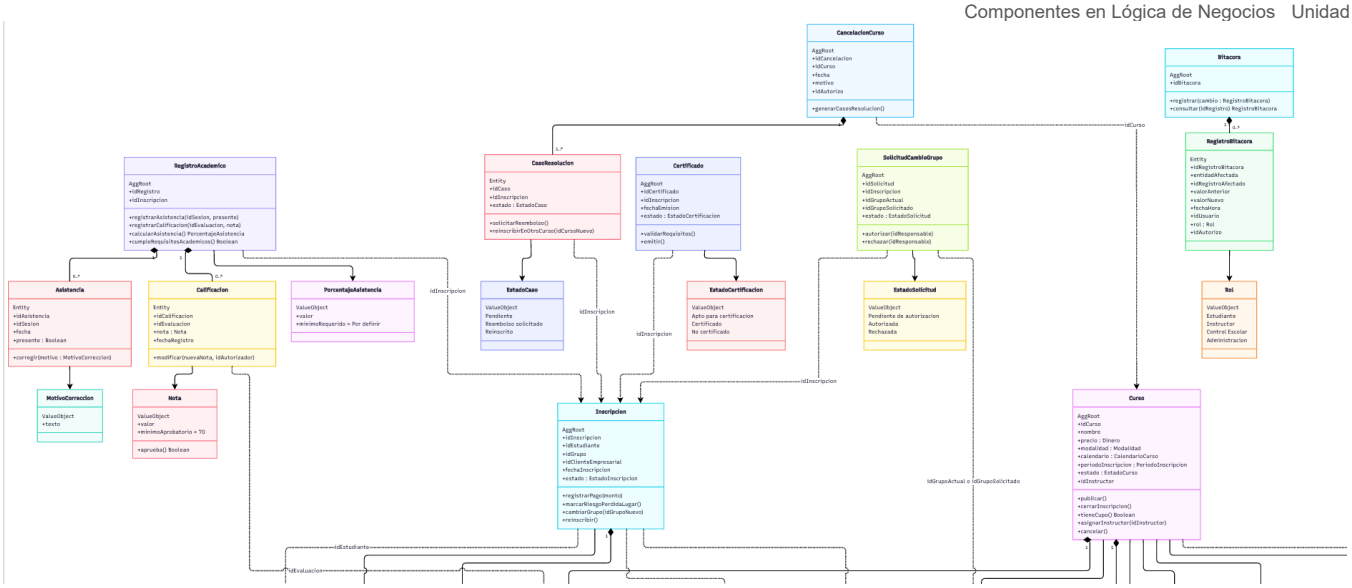


Diagrama de clases DDD de EduTech Academy - vista 3 de 4.

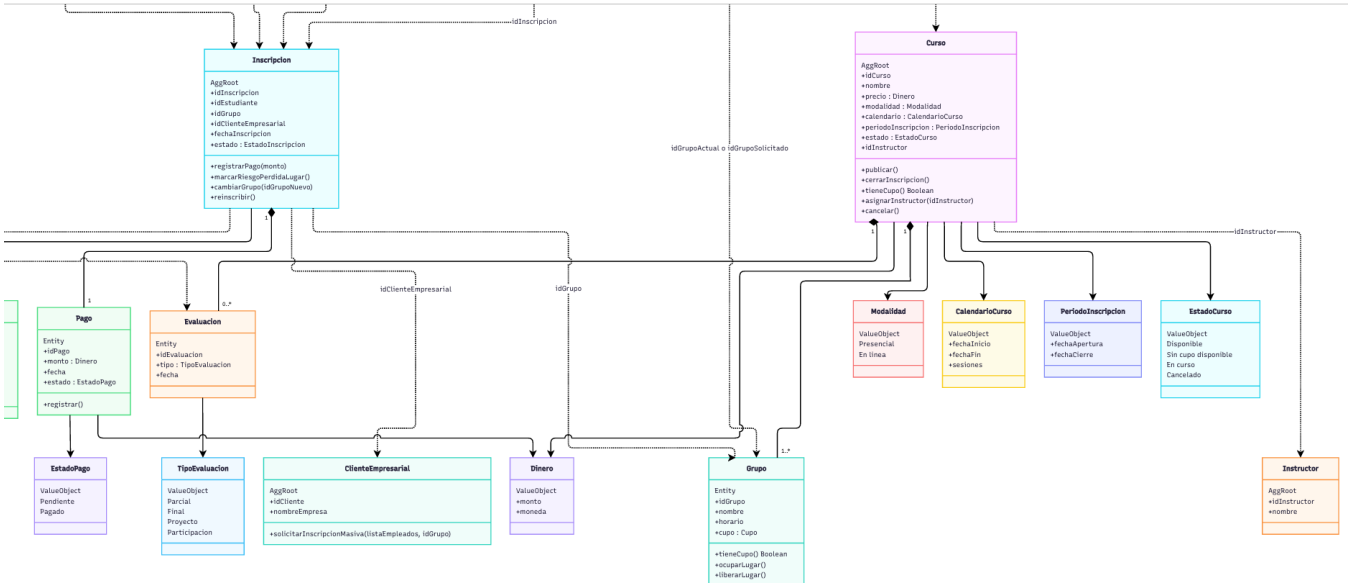


Diagrama de clases DDD de EduTech Academy - vista 4 de 4.

Diagramas de secuencia

Se documentan dos flujos representativos: certificación, como flujo principal sujeto a validación de criterios, y cancelación de curso, como flujo con resolución para las inscripciones afectadas.

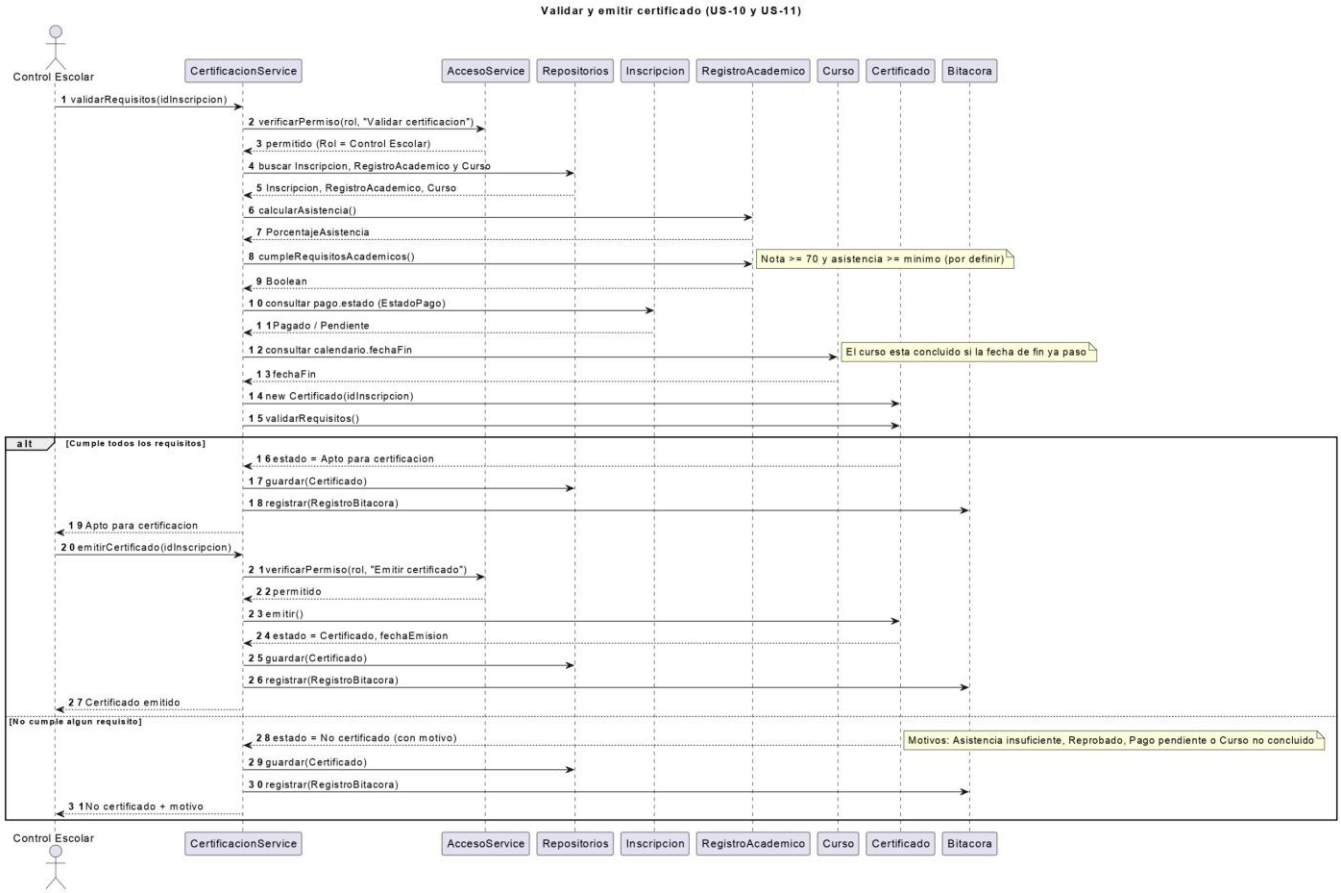


Diagrama de secuencia - flujo de certificación.

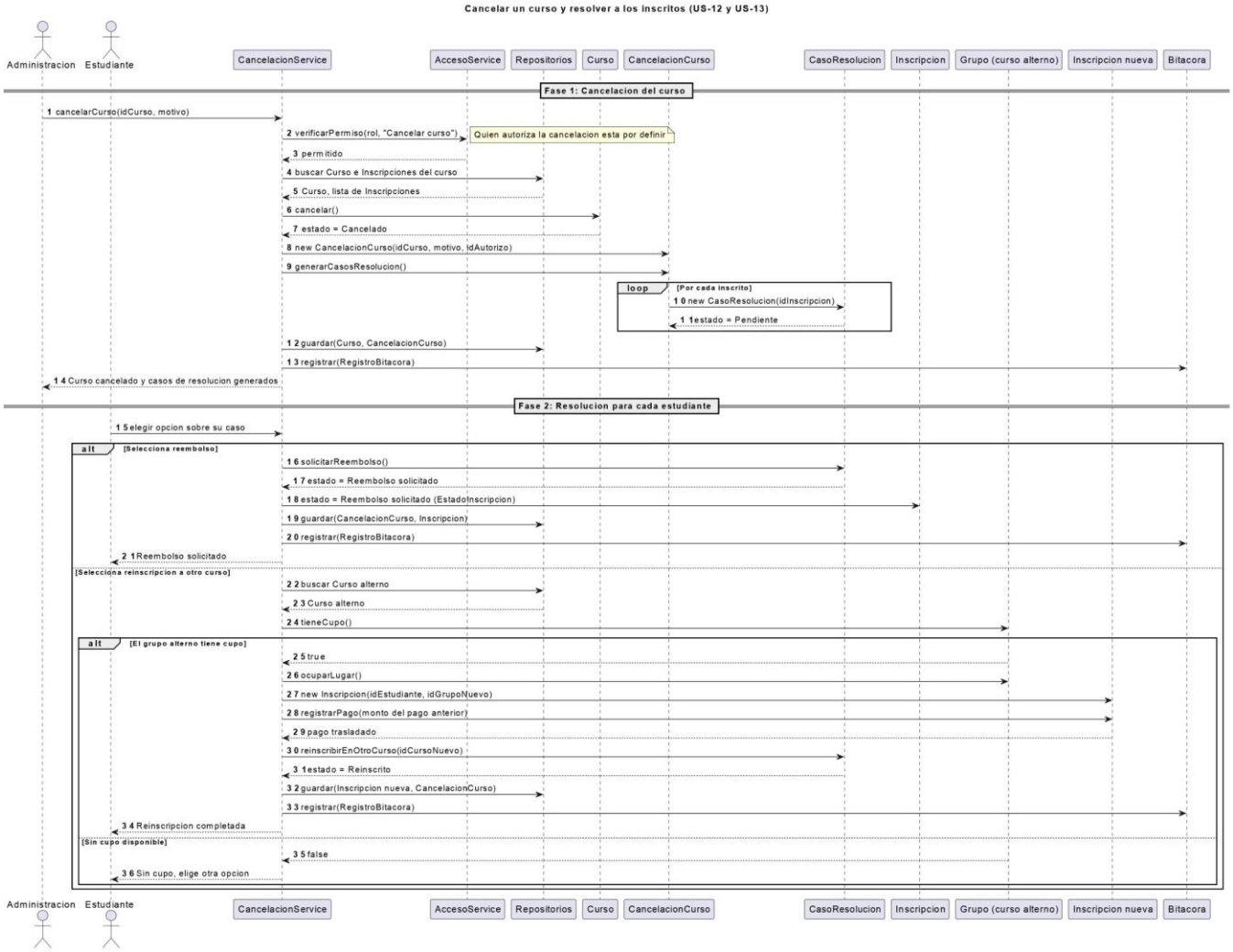


Diagrama de secuencia - flujo de cancelación de curso.

Architecture Decision Records (ADR)

Los ADR registran las decisiones arquitectónicas que tienen consecuencias relevantes para el diseño. Cada registro documenta contexto, alternativas, decisión, consecuencias y trazabilidad.

ADR-001	**Uso de Domain Model como patrón principal**
Descripción	EduTech Academy presenta reglas de negocio complejas relacionadas con certificación, control de cupos, estados del estudiante, pagos, cambios de grupo, modificación de calificaciones, reinscripciones y sustitución de instructor. Además, existen reglas que todavía pueden cambiar, como la asistencia mínima, el plazo de pago, el costo de reinscripción y el límite de cambios de grupo.
Decisión	Se utilizará Domain Model como patrón principal para representar la lógica de negocio del sistema. Las reglas serán encapsuladas dentro de entidades, aggregates y objetos de valor: los aggregates Curso, Inscripción, RegistroAcademico y Certificado, y objetos de valor como Cupo y Nota.
Alternativas consideradas	Transaction Script; Table Module; Active Record.
Consecuencias	Las reglas de negocio quedan centralizadas en un solo lugar, facilitando el mantenimiento, las pruebas y la evolución del sistema. Como desventaja, aumenta la complejidad inicial del diseño.
Trazabilidad	BRD (certificación y estados del estudiante); BR-03, BR-07; RNF-03; US-04, US-09, US-10, US-11.
ADR-002	**Uso de Service Layer para coordinar casos de uso**
Descripción	Los casos de uso de EduTech Academy requieren coordinar distintos objetos del dominio. Por ejemplo, una inscripción puede involucrar al estudiante, curso, grupo, inscripción y pago.
Decisión	Se implementará una Service Layer encargada de coordinar los casos de uso del sistema. Cada Application Service recuperará los objetos necesarios, invocará el comportamiento del dominio, persistirá los cambios y coordinará los eventos generados. La Service Layer no contendrá reglas de negocio propias.
Alternativas consideradas	Colocar la lógica directamente en controladores; utilizar un Transaction Script independiente para cada operación.
Consecuencias	Se logra una separación clara entre coordinación y reglas de negocio. Como consecuencia negativa, se agrega una capa adicional a la arquitectura.
Trazabilidad	US-01 a US-19 (casos de uso); US-19 y RNF-01 (permisos por rol); EX-01 a EX-04.

ADR-003	**Uso de Repository para acceso a persistencia**
Descripción	Los objetos del dominio deben almacenarse y recuperarse sin depender directamente de la base de datos.
Decisión	Se utilizará Repository como abstracción para guardar y recuperar los aggregates del dominio. La Service Layer utilizará los repositorios para acceder a la persistencia.
Alternativas consideradas	Active Record; acceso directo a la base de datos desde las entidades.
Consecuencias	El dominio queda desacoplado de la tecnología de persistencia y puede probarse con mayor facilidad. Como desventaja, se requieren interfaces y clases adicionales.
Trazabilidad	RNF-02 (integridad) y RNF-03; problema actual de información en archivos separados (BRD).
ADR-004	**Uso de Aggregates como límites de consistencia**
Descripción	Existen objetos relacionados que deben mantener reglas consistentes entre sí. Curso administra Grupo y Evaluación; Inscripción contiene Pago; RegistroAcademico contiene Asistencia y Calificación.
Decisión	Se utilizarán Aggregates con un Aggregate Root como punto principal de acceso para realizar modificaciones. Los objetos internos no deberán modificarse directamente desde fuera del aggregate.
Alternativas consideradas	Permitir modificaciones directas sobre entidades internas; modelar todas las entidades de forma independiente.
Consecuencias	Se protege la consistencia del modelo y se reduce el riesgo de cambios inválidos. Como desventaja, se deben definir cuidadosamente los límites de cada aggregate.
Trazabilidad	BR-03 y US-04 (cupo 5–25); RNF-02; US-05 (Inscripción y Pago); US-07 y US-08 (asistencia y calificaciones).

ADR-005	**Uso de Value Objects para conceptos sin identidad**
Descripción	Dentro del dominio existen conceptos definidos por sus valores y que no requieren identidad propia.
Decisión	Se modelarán como Value Objects conceptos como Cupo, Nota, Dinero, Modalidad, CalendarioCurso, PorcentajeAsistencia y los estados de curso, inscripción, pago, certificación, solicitud y caso de resolución.
Alternativas consideradas	Representar estos valores únicamente como tipos primitivos o campos simples dentro de las entidades.
Consecuencias	Las validaciones quedan encapsuladas dentro de cada concepto, se reduce la repetición de reglas y mejora la claridad del modelo.
Trazabilidad	Calificación mínima aprobatoria 70 y cupo 5–25 (BRD); BR-01, BR-06; US-04, US-08, US-10.
ADR-006	**Uso de Domain Events para comunicación entre procesos**
Descripción	Algunas acciones generan consecuencias en otras partes del negocio. Por ejemplo, cancelar un curso puede generar reembolsos, reinscripciones y registros en la bitácora.
Decisión	Se utilizarán Domain Events para representar hechos relevantes y permitir que otros componentes reaccionen sin acoplamiento directo. Entre ellos: InscripcionRealizada, PagoRegistrado, CursoCancelado, CalificacionModificada, CambioGrupoAutorizado y CertificadoEmitido.
Alternativas consideradas	Llamadas directas entre componentes; concentrar toda la coordinación dentro de la Service Layer.
Consecuencias	Se reduce el acoplamiento y se facilita la extensión del sistema. Como desventaja, puede resultar complejo seguir el flujo completo de una operación.
Trazabilidad	US-12 y US-13 (cancelación y resolución); US-09, US-11 y US-15; RNF-03.

ADR-007	**Bitácora de auditoría inmutable**
Descripción	EduTech Academy necesita mantener trazabilidad de operaciones sensibles como modificaciones de calificaciones, cambios de grupo, sustituciones de instructor y otras acciones importantes.
Decisión	Se implementará una Bitácora de Auditoría inmutable. Cada registro almacenará quién realizó la operación, cuándo ocurrió, el valor anterior, el valor nuevo y quién autorizó el cambio. Una vez creado, no podrá modificarse.
Alternativas consideradas	Auditoría manual dentro de cada operación; uso exclusivo de triggers en la base de datos.
Consecuencias	Se obtiene trazabilidad confiable y se facilita la identificación de cambios y responsables. Como desventaja, aumenta la cantidad de información almacenada.
Trazabilidad	RNF-03; BRD (Trazabilidad); US-07, US-09, US-15, US-17 y US-18.
ADR-008	**Parametrización de reglas de negocio aún no definidas**
Descripción	Algunas reglas todavía no han sido confirmadas por EduTech Academy, como asistencia mínima, plazo de pago, costo de reinscripción y límite de cambios de grupo.
Decisión	Estas reglas serán implementadas como valores configurables y no como valores fijos dentro del código.
Alternativas consideradas	Definir valores temporales como constantes; esperar a que todas las reglas estén definidas antes de desarrollar.
Consecuencias	El sistema podrá adaptarse cuando EduTech confirme las políticas definitivas, evitando modificar grandes partes de la lógica de negocio.
Trazabilidad	Reglas pendientes del BRD (asistencia mínima, fecha límite de inscripción, plazo de pago, costo de reinscripción); BR-02, BR-04; US-05, US-10 y US-16.

ADR-009	**Uso de consultas simples para operaciones de solo lectura**
Descripción	No todas las operaciones requieren utilizar todo el Domain Model. Algunas consultas solo leen información y no contienen reglas de negocio complejas, como el catálogo de cursos, los pagos pendientes y la consulta de bitácora.
Decisión	Para estas operaciones se permitirá utilizar consultas directas o un enfoque simple como Transaction Script, sin cargar todo el Domain Model.
Alternativas consideradas	Obligar a que todas las operaciones, incluso las consultas simples, utilicen el Domain Model como regla.
Consecuencias	Se reduce la complejidad y la sobrecarga en operaciones de solo lectura. Como consecuencia, no todas las operaciones utilizarán exactamente el mismo estilo arquitectónico.
Trazabilidad	US-01, US-06 y US-18; Etapa 6 (Queries).

Etapa 9 Bitácora de uso y validación de IA

La bitácora documenta el uso de IA como apoyo al análisis y registra la revisión humana aplicada a las propuestas. Se conservan el objetivo de la consulta, el prompt, el aporte útil, los errores o supuestos detectados y la decisión final del equipo.

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1	Comparar los tres patrones de lógica de negocio para EduTech.	(Carlos Hugo Garcia Gomez) Con base en las reglas de certificación, cupo 5–25 y cambios de grupo del BRD, compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.	Tabla con ventajas, riesgos y consecuencias de cada patrón; sirvió de borrador para la tabla comparativa.	Dio la ventaja a Domain Model sin citar evidencia del caso; se agregó la evidencia de la Unidad 2 (BRD, US y mapa de dominio).	Se eligió Domain Model y se redactó la justificación con el caso de EduTech.
2	Obtener candidatos a entidades, Value Objects, Aggregates y Domain Events.	(Carlos Hugo Garcia Gomez) Revisa este glosario y estas reglas de EduTech. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events e indica la evidencia y las dudas.	Lista inicial de candidatos y dudas, por ejemplo si Calificación es entidad o Value Object.	Propuso demasiadas entidades, una por cada tabla; se descartaron las que no tenían identidad propia.	Se definieron aggregates pequeños, Nota como Value Object y la Bitácora como aggregate aparte.
3	Proponer las firmas de los Application Services.	A partir de los casos de uso CU-01 a CU-05, propone firmas de Application Services. Separa la coordinación de las reglas de dominio y explica cualquier responsabilidad dudosa.	Borrador de servicios y métodos como registrarInscripcion y emitirCertificado.	Puso validaciones de cupo y certificación dentro del servicio; se movieron a Curso y Certificado.	Los servicios solo coordinan y las reglas quedan en el dominio.
4	Revisar el diseño con SOLID y armar los borradores de diagramas.	Analiza este diseño y señala posibles violaciones SOLID, sin reescribir todo. Después convierte la arquitectura validada en un borrador de diagramas sin agregar clases ni reglas nuevas.	Lista de hallazgos SOLID y borrador de los diagramas de dominio, clases y secuencia.	Algunos nombres no coincidían con el glosario y la asistencia mínima quedó como valor fijo.	Se corrigieron los nombres, la regla pendiente quedó configurable y los diagramas se revisaron a mano.

Etapa 10 Matriz de trazabilidad integral de la arquitectura

La matriz relaciona los casos de uso seleccionados con las historias de usuario, los componentes principales y las decisiones arquitectónicas que los soportan. Se construye únicamente con elementos ya documentados en la propuesta de EduTech Academy.

Caso / Necesidad	Historia(s)	Agregado / Componente	Decisión / ADR
CU-01 Consulta del catálogo de cursos	US-01	CatalogoService, CursoRepository, CursoDisponibleDTO	ADR-009
CU-02 Inscripción individual o empresarial	US-02, US-03	InscripcionService, Curso/Grupo, Inscripcion/Pago, UnitOfWork	ADR-001, ADR-002, ADR-003, ADR-004
CU-03 Gestión de curso y control de cupo	US-04, US-17	CursoService, aggregate Curso, Cupo, EventBus	ADR-001, ADR-004, ADR-005, ADR-006
CU-04 Registro académico y modificación de calificación	US-07, US-08, US-09	AcademicoService, RegistroAcademico, Nota, Bitacora	ADR-001, ADR-005, ADR-006, ADR-007
CU-05 Validación y emisión de certificado	US-10, US-11	CertificacionService, RegistroAcademico, Inscripcion, Curso, Certificado	ADR-001, ADR-002, ADR-004, ADR-006, ADR-008
Cambio de grupo	US-14, US-15	SolicitudCambioGrupo, Curso/Grupo, Bitacora	ADR-004, ADR-006, ADR-007
Cancelación y resolución	US-12, US-13	CancelacionCurso, CasoResolucion, Reembolso/Reinscripcion, Bitacora	ADR-006, ADR-007
Pagos pendientes / bitácora	US-06, US-18	Queries y modelos de lectura	ADR-009

Conclusiones

La propuesta de EduTech Academy adopta Domain Model como núcleo de la lógica de negocio y lo complementa con Service Layer, Repository y Unit of Work para separar coordinación, persistencia y transacciones. La arquitectura protege reglas críticas como cupo, estados, pagos, trazabilidad y certificación, y mantiene configurables aquellas políticas que aún requieren confirmación del negocio.

Los diagramas de dominio, clases y secuencia representan los mismos conceptos y flujos definidos en el documento. Los ADR dejan registro de las decisiones principales y la bitácora de IA documenta cómo las propuestas generadas fueron revisadas y corregidas por el equipo.